Ron Hensley

ronkhensley@gmail.com

Practice

Focus: Advanced Python I/O: Files, Command-Line Arguments, and Filters

Today
Hey Ron! This session is about digging deeper or exploring advanced topics in Python—think of it as leveling up your skills, not just reviewing what you already know.

Is there a topic or concept you want to dive into, or would you like me to suggest a few interesting areas we could explore together?

you can suggest some

Great! Here are a few advanced or deeper areas you might like:

Working with lists: using methods like append, remove, and sort, or slicing in creative ways
Mastering string manipulation: using methods like strip, split, replace, and indexing tricks
Writing and using functions: focusing on parameters, return values, or nesting functions for more power
Do any of these sound interesting, or would you like to go in a different direction?

sure although ive done a good bit of that in previous practices one advanced i actually would like to explore is reading from other inputs then keyboard ie storing and saivng then reading in from a text file for rpersistance across invookations and readng from stdin and right back out to stdout for building filters like filter.y | filter.py oh and reading in $args sent to the program and using them

That’s an awesome direction, Ron! You want to dig into:

Reading from and writing to text files for persistence
Handling standard input/output, like making your program behave like a filter
Accessing command-line arguments passed to your script
How much time would you like to spend on this deeper practice today? I recommend anywhere from 15–60 minutes.

60 is fine

Perfect! To confirm, your goal is to explore advanced input/output in Python for about 60 minutes, focusing on:

Reading/writing to text files
Using stdin/stdout for filtering
Reading command-line arguments
Does that sound right? Want to add or change anything before I lock it in?

yyes perfect

Awesome, your practice focus is set! I’m generating a session around file I/O, stdin/stdout, and command-line arguments now.

It’ll be ready in just a moment—thanks for your patience!

Here’s how we’ll roll today:

Start with a quick check that your Python setup is ready (REPL/terminal, basic input/output, etc).
Dive into reading from and writing to text files—making data persistent.
Try making a “filter” script: reading from stdin, writing to stdout.
Work with command-line arguments to make your scripts flexible.
Tie it all together with a combo challenge.
End with a quick reflection and next-step ideas.
Remember: you’re at the center! Want to adjust, slow down, or turn up the challenge? Just say so and I’ll flex the plan.

Mistakes happen, especially with file paths, arguments, or newlines. If something feels off, shout it out so we can fix it together.

Ready to start with the warm-up check?

im ready

Great! First, let’s confirm your Python environment is ready for local file and terminal work.

Can you open a terminal or command prompt, launch the Python REPL with python or python3, type print(input()), press Enter, and enter some text when prompted?

Let me know once you’ve done that, or if you need help opening a terminal or running Python.

got it

Nice! Last check—exit the Python REPL (usually with exit() or pressing Ctrl+D/Ctrl+Z). Then, in your terminal, run:

Bash

python --version
or

Bash

python3 --version
Let me know what you see!

Python 3.12.10
i alo have
pythng 3.14.6 installe as py

Perfect, Ron! You’re set up with Python 3.12 and 3.14—plenty for everything we’ll do.

Let’s move to our first deep dive: file input and output.

Would you like to jump in with zero hints and see how far you get? Or want a short guide before you code? Your call!

i coulld self lookup ut id apreciate a quick into to the unkwons like what to import for file reads and for stdin import os is myt guess

Great mindset! Here’s a quick intro:

For file reading/writing, you don’t need any import—Python’s built-in open() handles it.
For stdin/stdout (reading lines from the terminal/input stream), import sys and use sys.stdin and sys.stdout.
You don’t need os for basic file or stdin/stdout work.
Want a reminder on the with open('file.txt') as f: pattern before you jump in? Or feeling good to try from here?

reminder pls

Absolutely! Here’s the pattern:

Python

```
with open('filename.txt', 'mode') as file:
    # do something with file, like file.write() or file.read()
```

The with block handles opening and closing the file for you.
'w' mode: write (overwrites file)
'a' mode: append (adds to file)
'r' mode: read (opens for reading)
Inside, just use file.write() or file.read(). Want to give it a try now?

sure thing

Awesome! Here’s your first hands-on:

Create a file called notes.py. Make it so that:

It appends one new line to notes.txt each time you run it. The new line should have the current time.
Then, it reopens notes.txt and prints all lines to the screen.
Try it with the with open(...) as ... pattern for both append and read modes.

When you’re ready, use Run & Submit to show me your code and output. If you get stuck, just say so!

question on how o iterate through the file using reasds and deltect EOF and exit that iteration

Good question! When you open a file for reading, you can iterate through it line by line using a simple for-loop:

Python

```
with open('notes.txt', 'r') as file:
    for line in file:
        print(line, end='')
```

This loop automatically stops at the end of the file—no need to detect EOF yourself.
If you use file.read() or file.readline(), you’d need to check for an empty string (which means EOF).
But usually, stick with for line in file:—it’s clean and Pythonic! Ready to try your notes.py script?

close but getting extara line feed

That’s a common one! File lines usually end with a newline already, so when you use print(line), print() adds another newline.

To avoid double-spacing, print like this:

Python

```
print(line, end='')
```

This keeps each line as it is from the file. Try updating your print statement and run again!

i tried that and get undefined end

Ah, if you get “undefined end,” make sure you’re using the correct syntax for print() in Python:

Python

```
print(line, end='')
```

The comma is important—end='' should be an argument to the print() function, not a statement by itself.

Double-check for typos or misplaced colons.

Try this version and use Run & Submit so I can see your code and output. If you still get an error, submit anyway and I’ll spot it!

Python

```
from datetime import datetime
end = str(datetime.now())
with open("notes.txt","a") as file:
    file.write(f"{datetime.now()}\n")

with open("notes.txt","r") as file2:
    for line in file2:
        print(line, end= "")
```

Nice work, Ron! Your code is solid—appends a timestamp each run and prints the full contents of notes.txt without extra newlines.

You handled both file modes correctly and used the end='' fix just right. ☑

Want to run it a second time and check that a new timestamp does get added to the file? If so, use Run & Submit again with your code.

Or, if you're confident it's working, I'll explain anything you want before we move to stdin/stdout "filter" programs. Your call!

i tested on my local pythomng command line and it worked hre im not srure the file produiced is pesistant

Exactly, Ron. On local Python, your file updates and persists between runs.

Here, the platform's file system may reset or not preserve files, so persistence is limited.

But you did everything right in your code and process!

Ready to try some stdin/stdout filter magic next?

sure please folow up on the print filter though whats end= "" doing exactly

Great follow-up!

When you use print(line, end=''), the end='' part tells Python not to add a newline after printing each line.

By default, print() adds a \n at the end.

But each line from the file already ends with a newline, so using end='' avoids double-spacing.

That's why your output stays single-spaced.

Ready to move on to writing a filter that reads from stdin and outputs to stdout?

yeah i foound in tests if i didnt use \n in the write it al went to the same line

Exactly! If you leave out the \n in the write(), every timestamp just gets glued together on one line.

So, always add \n at the end when writing separate lines.

Great experiment!

Let's jump to building your first filter.

Ready to practice reading from stdin, converting to uppercase, and outputting?

sure

Awesome! Here's the task:

Create a script called upper.py that:

Reads from standard input (stdin)
Converts every line to uppercase
Writes each result to standard output (stdout)
Hint:

You'll want import sys
Loop: for line in sys.stdin:
Use print(line.upper(), end='')
After you code it, use Run & Submit so I can see your script and output. If you get stuck, let me know!

it runs on my local python fine but here its getting OSError:{Errho 29]I/O error

Great job testing locally—your code is correct. That error happens because this online environment can't pipe or truly emulate stdin from the terminal like your machine can.

So, here you'll see I/O errors for scripts meant for true piping or interactive input.

But your solution works as intended!

Want to see a sample code and do one quick practice, or ready to move on to command-line arguments?

```
import sys
empty_line = False
while not empty_line:
for line in sys.stdin:
if line == "\n":
empty_line = True
break
print(line.upper(), end= "")
```

That's clever, Ron! You're reading from sys.stdin, checking for an empty line ("\n"), and converting the rest to uppercase.

The loop will end when the user submits a blank line, thanks to your empty_line flag.
print(line.upper(), end="") still keeps output tidy, just like before.
This is a solid approach for handling streams or "end on an empty line" logic.

If you want another quick filter challenge, I can give you one—

or do you want to try playing with command-line arguments next?

another is fine reminds me of programming in the 90s doing useful filters

Love that! Filters are classic and powerful.

Here's your next mini-challenge:

Write a script called grep_like.py that:

Takes a substring as a command-line argument
Reads lines from stdin
Prints only the lines that contain that substring
Tip:

Check sys.argv for the substring
For each line in sys.stdin, use if substring in line: to filter
Once done, use Run & Submit! If you want, try with different substrings.

Python

```
import sys

if len(sys.argv) < 2:
    print("Error: Missing arguments!")
    print(f"Usage: python {sys.argv[0]} [pattern]")
```

```
    sys.exit(1)


std_input = sys.argv[1]


for line in sys.stdin:
    if line == "\n":
        break
    elif line == "":
        break
    if std_input in line:
        print(line, end= "")
```

Great job handling argument checks and the filter logic, Ron! Your code correctly:

Checks for the substring argument
Breaks on empty input
Prints lines containing the substring, without adding extra newlines
A couple of feedback points:

Your filter compares if line == "\n": to stop at an empty line, which works but is optional—most filters just finish when stdin closes (like piping from a file). So you could let the loop simply run out, but your way is more interactive for manual entry.
The variable name std_input is fine, but substring or pattern would clarify its purpose.
Try this mini-tweak:

Change your script to not break on "\n" or "" (so it just consumes all input) and test piping a file in your local setup, if curious.

Ready for a final applied combo, or want to tweak this filter more?

also fails in this environment i had to google hwo to check EOF though for
cat file.txt | pytgon grep_like.py
or i would hand forevver

Exactly—since this environment doesn't support true stdin piping, your script can't fully run as intended here. On your own machine with cat file.txt | python grep_like.py pattern, it would work and finish at EOF (end-of-file).

Nice job googling about EOF! In most cases, for line in sys.stdin: just stops when there's no more input (EOF). No need to break—unless you want to end with a specific user signal, like a blank line.

Even though we can't fully demo piping here, your approach is exactly how classic UNIX-style filters work.

Want to combine everything into a script that:

Reads from a file,
Filters by substring,
Writes results to another file,
And prints a summary line?
Or, would you like to experiment more with stdin filters?

print a summary line of what? And would take the file to read as an argument OR stidin , and write to filename given another argument OR to stdout ? just thinking that would make it truly useful

Great thinking! The summary line could say something like:

"Saved X lines to outfile" (if writing to a file), or "Matched X lines" (if writing to stdout).

To make it truly flexible, your script could:

Accept both an input filename or read from stdin (if filename is '-' or missing)
Output to an output file, or stdout (if output filename is '-' or missing)
Filter lines that contain a given substring (from args)

Want to build that together step by step? Or start with a version where both input and output are files (easy first, flexible second)?

ill do that

Awesome choice!

Here’s a step-by-step outline for an “applied combo” script:

Call it persist_filter.py
Arguments: substring infile outfile
Reads lines from infile
Writes matching lines (those containing substring) to outfile
At the end, prints: Saved X lines to outfile
You can reuse logic from before for reading files, filtering lines, and counting matches.

Give it a try, use Run & Submit, and let me know if you want any hints!

Python

```
import sys
import argparse
std_input = ""
std_output = ""

#Parse stidin or passed file name
def parse_file(in_file, patt):
    text_file = []
    if in_file:    #if we were given a valid file name, use it
        with open(f"{in_file}","r") as file:
            for line in file:
                if line == "\n":   #check for blank lines or
                   break
                elif line == "":   #end of file and exit if found
                    break
                if patt in line:   #still here? Only store lines containing the search
                pattern
                    text_file.append(line)
    else:  #No file name was give, read stdin (Keyboard)
        for line in sys.stdin:
            if line == "\n":       #check for blank lines or
                break
            elif line == "":       #end of file and exit if found
                break
            if patt in line:       #still here? Only store lines containing the search
            pattern
               text_file.append(line)
    return text_file               #Send back the file parsed to only include lines
    containing pattern

#write file contents to stdout (screen) or passed file name
def print_outfile(loc_file, std_outfile):
    if std_outfile: #if we were given a valid file name, use it
        with open(f"{std_outfile}","w") as file:
            for line in loc_file:
                if line == "\n":   #check for blank lines or
                   break
                elif line == "":   #end of file and exit if found
                    break
                file.write(line)
    else:  #No file name was give, write to stdout (Screen)
        for line in loc_file:
            if line == "\n":       #check for blank lines or
                break
            elif line == "":       #end of file and exit if found
                break
            print(line, end= "")
```

```
#Parse command line for search pattern (mandatory) and optionally
#input file name and/or output file name
parser = argparse.ArgumentParser(description="Persistant Grep Filter")
parser.add_argument("pattern", help="<search pattern>")
parser.add_argument("-i","--infile", help="--infile <File Name>", default="")
parser.add_argument("-o", "--outfile", help="--outfile <File Name>", default="")


args = parser.parse_args()
pattern = args.pattern              #Store the search pattern
if args.infile:
    std_input = args.infile         #Store in input file name if given
if args.outfile:
    std_output = args.outfile       #Store the output file name if given

#Grep style search of the file or stdinput based on search pattern
read_file = parse_file(std_input, pattern)

#print grepped file conents to an output file or stdout
print_outfile(read_file, std_output)
```

Great effort, Ron! Your script is robust, using argparse for flexible options and separating reading and writing cleanly. Here's some quick feedback and help:

Platform Behavior:

The "SystemExit: 2" error typically means argparse did not find the required positional argument—the pattern. This can happen because the test environment may not support passing custom arguments, which is why you see the error.

Your Approach:

Reading files and storing matching lines with if patt in line is correct.
Writing to a file or stdout works as expected on your machine.
Handling blank/empty lines with checks is careful, but for non-interactive files you usually don't need to break on "\n"—just let the loop finish unless you expect interactive stops.

Summary Line:

Your code doesn't print the summary "Saved X lines…" after writing. That's the last requirement for this combo.

Quick suggestion:

After print_outfile(read_file, std_output), add:

Python

```
print(f"Saved {len(read_file)} lines to {std_output if std_output else 'stdout'}")
```

Want to update and submit with that summary line? Or would you like more tips on handling missing arguments/EOF?

```
import sys
import argparse
std_input = ""
std_output = ""

#Parse stidin or passed file name
def parse_file(in_file, patt):
text_file = []
if in_file: #if we were given a valid file name, use it
with open(f"{in_file}","r") as file:
for line in file:
if line == "\n": #check for blank lines or
break

elif line == "": #end of file and exit if found
break

if patt in line: #still here? Only store lines containing the search pattern
text_file.append(line)
else: #No file name was give, read stdin (Keyboard)
for line in sys.stdin:
```

```
if line == "\n": #check for blank lines or
break
elif line == "": #end of file and exit if found
break
if patt in line: #still here? Only store lines containing the search pattern
text_file.append(line)
return text_file #Send back the file parsed to only include lines containing pattern

#write file contents to stdout (screen) or passed file name
def print_outfile(loc_file, std_outfile):
if std_outfile: #if we were given a valid file name, use it
with open(f"{std_outfile}","w") as file:
for line in loc_file:
if line == "\n": #check for blank lines or
break
elif line == "": #end of file and exit if found
break

file.write(line)
else: #No file name was give, write to stdout (Screen)
for line in loc_file:
if line == "\n": #check for blank lines or
break
elif line == "": #end of file and exit if found
break

print(line, end= "")
print(f"Saved {len(loc_file)} lines to {std_outfile}")

#Parse command line for search pattern (mandatory) and optionally
#input file name and/or output file name
parser = argparse.ArgumentParser(description="Persistant Grep Filter")
parser.add_argument("pattern", help="<search pattern>")
parser.add_argument("-i","--infile", help="--infile <File Name>", default="")
parser.add_argument("-o", "--outfile", help="--outfile <File Name>", default="")

args = parser.parse_args()
pattern = args.pattern #Store the search pattern

if args.infile:
std_input = args.infile #Store in input file name if given
if args.outfile:
std_output = args.outfile #Store the output file name if given

#Grep style search of the file or stdinput based on search pattern
read_file = parse_file(std_input, pattern)

#print grepped file conents to an output file or stdout
print_outfile(read_file, std_output)
```

This is an excellent, full-featured filter script, Ron! You made it flexible with both file and stdin/stdout options and added the summary line as requested.

Highlights:

Uses argparse for clean argument handling
Reads and filters lines using both file or stdin
Writes to either a specified file or stdout
Prints a summary of how many lines matched
What could be improved for future code?

In most file-processing scripts, there’s no need to break on "\n" (blank line)—just let the loop go to EOF.
When outputting to stdout, the summary line currently always prints “to <filename>”. You could tweak it to say “stdout” if no output file.
But your solution is correct and very practical—great work!

How did all this feel? Want to reflect on each mode (file, stdin/stdout, arguments) now, or need help with anything still unclear?

i debated on that myself thinking only stdin needed it else use would need a keystroke to end input

Exactly—that’s a great insight! For real files, you just read to the end (EOF happens automatically). For stdin entered by hand, a signal (like Ctrl+D or Ctrl+Z) is needed to finish input, so some folks use a blank line as a friendlier end marker.

You handled the differences well and your reasoning is solid.

Ready for a fast reflection on what you now know about files, filters, and arguments? Or anything else you want to re-explore before wrapping up?

about ready to wrap it up i learned a tone went from no idea to how to read files, wrote to ffilese, parse argulements to a good understanding and working projects to refer to

That’s fantastic, Ron! You made a huge leap—from zero to building real filter tools with files, stdin/stdout, and command-line arguments.

You now know:

How to read and write files the Pythonic way
How to build classic filters using stdin/stdout
How to make scripts flexible with arguments
You’ve also got some great “real” scripts you can reuse anytime!

Are you satisfied and ready to finish for now?

very thank you

You did awesome today, Ron. I’m proud of your progress and your curiosity!

Whenever you want to expand or apply these skills again, just let me know. Have a great rest of your day!